

Introduction to Spring Framework

Introduction to the Spring Framework

- The **Spring Framework** is a comprehensive, lightweight, and modular solution designed for building enterprise-ready applications.
- Its modular architecture allows developers to use only the components they need, ensuring flexibility and efficiency.
- Spring is a **Dependency Injection** framework to make Java applications loosely coupled.
- Key Features:
 - **Lightweight and Modular:** Developers can integrate individual components without needing the entire framework.
 - **Inversion of Control (IoC):** Uses Dependency Injection (DI) to manage dependencies efficiently.
 - **Aspect-oriented Programming (AOP):** Allows for clean separation of concerns, making applications more maintainable.
 - **Data Access and Persistence Support:** Provides integration with JDBC, Hibernate, JPA, and other persistence technologies.
 - **Spring MVC Framework:** A powerful Model-View-Controller (MVC) framework for web applications.

Need for the Spring Framework

- The **Spring Framework** was developed to address the limitations of traditional Java Enterprise Edition (Java EE) development.
- Before Spring, Java developers faced several challenges:
- **Complexity of Java EE**
 - Traditional Java EE applications required extensive configuration and were tightly coupled with the application server.
 - Managing Enterprise JavaBeans (EJBs) was cumbersome and required complex setup.
- **Tight Coupling and Boilerplate Code**
 - Java applications often contained redundant boilerplate code for tasks like database access and transaction management.
 - Dependencies between components were hardcoded, making maintenance difficult.

How Spring Solves These Challenges

- **Simplifies Java Development:**

- Uses Dependency Injection (DI) to reduce code complexity.
- Eliminates the need for explicit object creation, making code more flexible.

- **Lightweight Alternative to EJB:**

- Provides features like transaction management and declarative configuration without requiring EJBs.
- Supports Plain Old Java Objects (POJOs), making applications easier to develop and test.

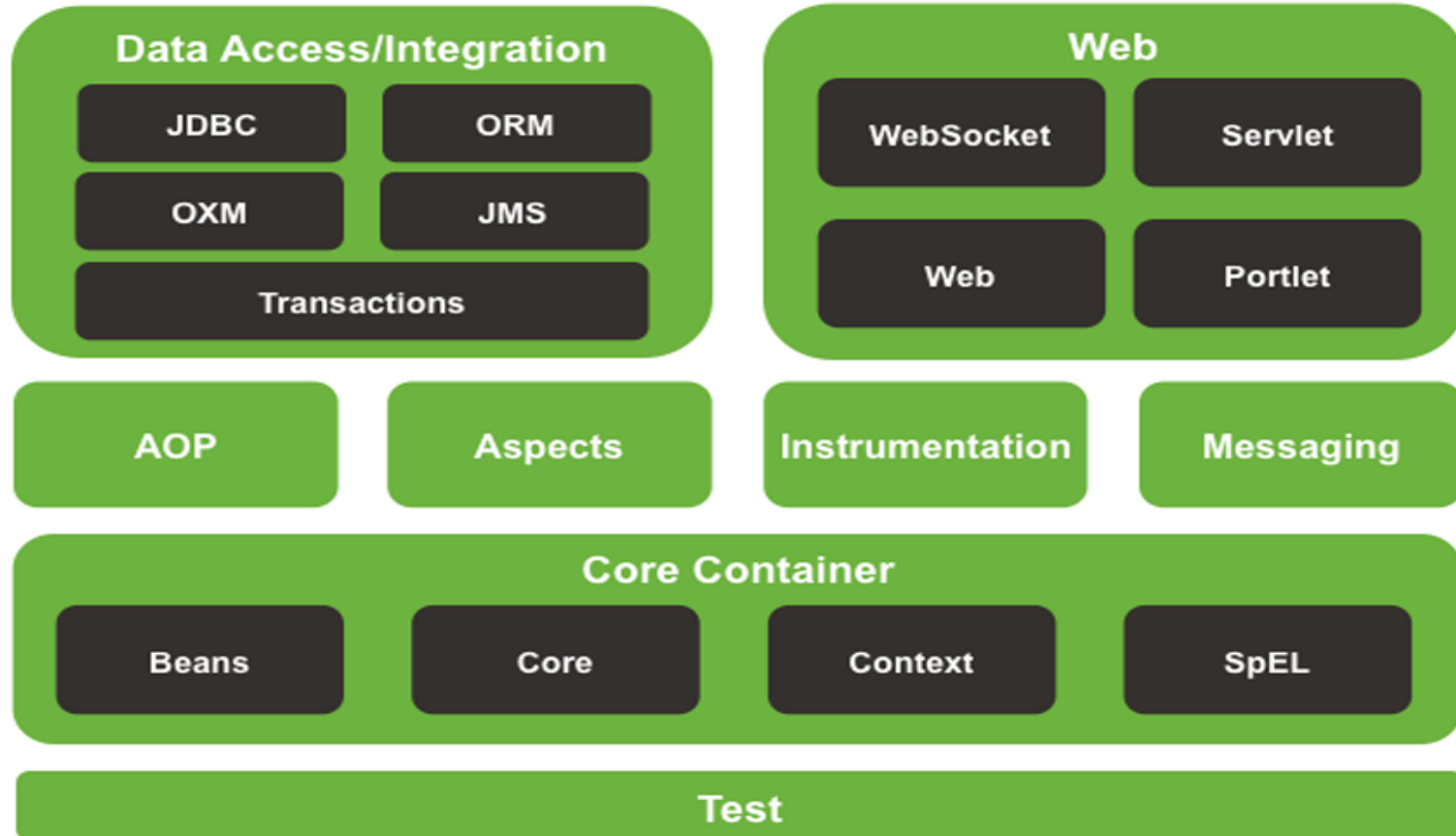
- **Better Integration & Modularity:**

- Works seamlessly with JDBC, JPA, Hibernate, and other technologies.
- Developers can use only the required Spring modules without unnecessary overhead.

Overview of Spring Framework:



Spring Framework Runtime



Overview of Spring Framework:

- **Core Container:**

- The Core Container consists of the **spring-core**, **spring-beans**, **spring-context**, **spring-context-support**, and **spring-expression** (Spring Expression Language) modules.
- The **spring-core** and **spring-beans** modules provide the fundamental parts of the framework, including the IoC and Dependency Injection features.
- The **BeanFactory** is a sophisticated implementation of the factory pattern.
- It removes the need for programmatic singletons and allows you to decouple the configuration and specification of dependencies from your actual program logic.
- The **Context** (spring-context) module builds on the solid base provided by the Core and Beans modules: The Context module inherits its features from the Beans module.
- The **ApplicationContext** interface is the focal point of the Context module.
- The **spring-expression** module provides a powerful Expression Language for querying and manipulating an object graph at runtime.

How Spring Solves These Challenges

- The AOP module equips Spring to support the implementation of aspects within your web application.
- Similar to Dependency Injection (DI), AOP promotes loose coupling of application objects.
- It enables you to separate the application logic from system-level services, known as cross-cutting concerns, such as transaction management and security.
- Consequently, this module serves as a foundation for implementing your own aspects in a Spring-enabled application.
- The Spring AOP module enables you to apply declarative enterprise services, such as transaction management, security, and logging, as well as messaging services, to your applications using XML configuration files.
- Since these services aren't an integral part of the application logic, the Spring container applies them transparently at runtime.
- You only need to specify how and where the container should apply these services.
- Additionally, you can implement your custom aspects in your application.

The IoC container:

- **IoC** also known as dependency injection (DI) is a process whereby objects define their dependencies.
- That is, the other objects they work with, only through constructor arguments, arguments to a factory method, or properties that are set on the object instance after it is constructed or returned from a factory method.
- The container then injects those dependencies when it creates the bean.
- The Spring framework provides the following types of containers:
 - **Bean factory**
 - **Application context**

How Spring Solves These Challenges

- The **BeanFactory** interface provides an advanced configuration mechanism capable of managing any type of object.
- We can use the bean factory container for creating simple Spring-enabled applications, where the focus is on providing support for dependency injection and dependency lookups.
- **ApplicationContext** is a subinterface of BeanFactory. It adds easier integration with Spring's AOP features;
- It is an advanced container that provides several enterprise-level functionalities, some of which are:
 - Providing a means for resolving text messages, including support for the internationalization of those messages.
 - Providing a generic way to load file resources, such as images.
 - Publishing events to beans, which are registered as listeners.

How Spring Solves These Challenges

- **ApplicationContext** is a subinterface of BeanFactory. It adds easier integration with Spring's AOP features;
- It is an advanced container that provides several enterprise-level functionalities, some of which are:
 - Providing a means for resolving text messages, including support for the internationalization of those messages.
 - Providing a generic way to load file resources, such as images.
 - Publishing events to beans, which are registered as listeners.
- It inherits all the capabilities of the BeanFactory interface, such as creating the beans, associating them with each other, and dispersing them to the users as per requests.

How Spring Solves These Challenges

- The Spring framework provides the following commonly used implementations of the application context container:
- **ClassPathXmlApplicationContext**: It is used to load the bean configuration file located at the specified classpath.
- **FileSystemXmlApplicationContext**: It is used to load the bean configuration file available in the specified file system.